## Unit 4 Progress Check: FRQ Part A

**1.** The Shelf class is used to store information about available spare car parts on each shelf in a shop. Each shelf contains a number of axles and a number of wheels, which are two types of car parts. A partial declaration of the Shelf class is shown.

```
public class Shelf
{
  /**
   *  Returns the number of axles on the shelf,
   *  guaranteed to be 0 or more
   */
  public int getNumAxles()
  { /* implementation not shown */ }

  /**
   *  Returns the number of wheels on the shelf
   *  guaranteed to be 0 or more
   */
  public int getNumWheels()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The Shop class maintains an array named shelfArray that represents all the shelves in the shop. A partial declaration of the Shop class is shown.

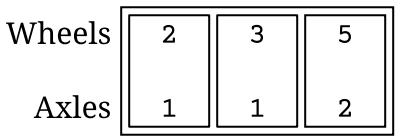
```
public class Shop
{
  /** An array of all shelves at the shop */
  private Shelf[] shelfArray;

  /**
   *  Returns the number of assemblies that can be made from
   *  all the parts on all the shelves. Each assembly requires
   *  2 wheels and 1 axle.
   *  Preconditions: shelfArray is not null.
   *                 no elements of ShelfArray are null.
   *  Postcondition: shelfArray is unchanged.
   */
  public int getPossibleAssemblies()
  { /* to be implemented */ }
}
```

## Unit 4 Progress Check: FRQ Part A

An assembly is a car part built using 2 wheels and 1 axle. Write the Shop method getPossibleAssemblies, which should return the maximum number of assemblies that can be made from the available wheels and axles from all shelves.

For example, suppose shelfArray contains the following Shelf objects.




For these contents of shelfArray, the method getPossibleAssemblies() should return 4. There are a total of 10 wheels and 4 axles. Since each assembly requires 2 wheels and 1 axle, there are 4 assemblies possible.

Complete method getPossibleAssemblies.

```
/**
 * Returns the number of assemblies that can be made from
 * all the parts on all the shelves. Each assembly requires
 * 2 wheels and 1 axle.
 * Preconditions: shelfArray is not null.
 *                no elements of ShelfArray are null.
 * Postcondition: shelfArray is unchanged.
 */
public int getPossibleAssemblies()
```

### Point 1

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all elements in `shelfArray` (no bounds errors)

**Canonical Solution:**

```
public int getPossibleAssemblies()
{
   int totalWheels = 0;
   int totalAxles = 0;

   for (Shelf p : shelfArray)
   {
```

## Unit 4 Progress Check: FRQ Part A

```
      totalWheels += p.getNumWheels();
      totalAxles += p.getNumAxles();
   }

   int wheelPairs = totalWheels / 2;
   if (wheelPairs < totalAxles)
   {
      return wheelPairs;
   }
   else
   {
      return totalAxles;
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to use the accessed value
- return early, as long as bounds and indices otherwise support accessing all elements

Responses will not earn the point if they

- use an indexed loop with correct bounds but fail to access elements inside the loop
- have a bounds error
- access the elements of the list incorrectly

**Point 2**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getNumAxles` and `getNumWheels` on an element from the array

**Canonical Solution:**

```
public int getPossibleAssemblies()
{
   int totalWheels = 0;
```

## Unit 4 Progress Check: FRQ Part A

```
   int totalAxles = 0;

   for (Shelf p : shelfArray)
   {
      totalWheels += p.getNumWheels();
      totalAxles += p.getNumAxles();
   }

   int wheelPairs = totalWheels / 2;
   if (wheelPairs < totalAxles)
   {
      return wheelPairs;
   }
   else
   {
      return totalAxles;
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- call either method incorrectly

**Point 3**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Computes total number of wheels and total number of axles

**Canonical Solution:**

```
public int getPossibleAssemblies()
{
   int totalWheels = 0;
   int totalAxles = 0;

   for (Shelf p : shelfArray)
```

## Unit 4 Progress Check: FRQ Part A

```
   {
      totalWheels += p.getNumWheels();
      totalAxles += p.getNumAxles();
   }

   int wheelPairs = totalWheels / 2;
   if (wheelPairs < totalAxles)
   {
      return wheelPairs;
   }
   else
   {
      return totalAxles;
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to identify all call parts
- return early

Responses will not earn the point if they

- fail to call `getNumAxles` or `getNumWheels`

**Point 4**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Returns count of available assemblies (algorithm)

**Canonical Solution:**

```
public int getPossibleAssemblies()
{
   int totalWheels = 0;
   int totalAxles = 0;
```

## Unit 4 Progress Check: FRQ Part A

```
   for (Shelf p : shelfArray)
   {
      totalWheels += p.getNumWheels();
      totalAxles += p.getNumAxles();
   }

   int wheelPairs = totalWheels / 2;
   if (wheelPairs < totalAxles)
   {
      return wheelPairs;
   }
   else
   {
      return totalAxles;
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- stop loop early or late due to incorrect bounds

Responses will not earn the point if they

- fail to return the computed value
- return an incorrect value due to early return

## Unit 4 Progress Check: FRQ Part A

**2.** The Surface class is used to store information about a flat rectangular surface. A partial declaration of the Surface class is shown.

```
public class Surface
{
  /**
   *  Returns the width of the surface, in feet
   */
  public double getWidth()
  { /* implementation not shown */ }

  /**
   *  Returns the length of the surface, in feet
   */
  public double getLength()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The Painter class maintains an array of surfaces that need to be painted.

```
public class Painter
{
  /** The array of surfaces to be painted */
  public Surface[] surfaceArray;

  /**
   *  Returns true if numGallons is enough to
   *  paint all surfaces in surfaceArray and
   *  returns false otherwise
   *  Preconditions: numGallons > 0
   *                 No elements of surfaceArray are null.
   *                 surfaceArray is not empty.
   *  Postcondition: surfaceArray is unchanged.
   */
  public boolean hasEnoughPaint(int numGallons)
  { /*to be implemented */ }
}
```

Write the Painter method hasEnoughPaint. The method should return true if numGallons gallons of paint is enough to paint all surfaces in surfaceArray. The method should return false otherwise. Each gallon of paint is enough to paint an area of 400 square feet.

## Unit 4 Progress Check: FRQ Part A

For example, suppose surfaceArray has the following contents.

| | | | | | |
|---|---|---|---|---|---|
| Length | 15.0 | 7.0 | 12.0 | 20.0 | 4.0 |
| Width | 5.0 | 10.5 | 10.0 | 20.0 | 11.5 |

In this example, the total area of all the surfaces in surfaceArray is 714.5 square feet.

- The call hasEnoughPaint(2) should return true because 2 gallons is enough paint for up to 800 square feet.
- The call hasEnoughPaint(1) should return false because 1 gallon is only enough paint for up to 400 square feet.

Complete method hasEnoughPaint.

```
/**
 * Returns true if numGallons is enough to
 * paint all surfaces in surfaceArray and
 * returns false otherwise
 * Preconditions: numGallons > 0
 *                No elements of surfaceArray are null.
 *                surfaceArray is not empty.
 * Postcondition: surfaceArray is unchanged.
 */
public boolean hasEnoughPaint(int numGallons)
```

**Point 1**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all elements in `surfaceArray` (no bounds errors)

**Canonical Solution:**

```
public boolean hasEnoughPaint(int numGallons)
{
   double totalArea = 0.0;
   for (Surface s : surfaceArray)
   {
      totalArea += s.getLength() * s.getWidth();
   }
```

## Unit 4 Progress Check: FRQ Part A

```
   if (totalArea <= numGallons * 400)
   {
      return true;
   }
   else
   {
      return false;
   }
}
```



| 0 | 1 |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to use the accessed value
- return early, as long as bounds and indices otherwise support accessing all elements

Responses will not earn the point if they

- use an indexed loop with correct bounds but fail to access elements inside the loop
- have a bounds error
- access the elements of the list incorrectly

**Point 2**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getWidth` and `getLength` on an element from the array

**Canonical Solution:**

```
public boolean hasEnoughPaint(int numGallons)
{
   double totalArea = 0.0;
   for (Surface s : surfaceArray)
   {
      totalArea += s.getLength() * s.getWidth();
   }
```

## Unit 4 Progress Check: FRQ Part A

```
   if (totalArea <= numGallons * 400)
   {
      return true;
   }
   else
   {
      return false;
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses will not earn the point if they

- call either method incorrectly

**Point 3**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Determines whether identified total area is less than or equal to 400 times the number of gallons

**Canonical Solution:**

```
public boolean hasEnoughPaint(int numGallons)
{
   double totalArea = 0.0;
   for (Surface s : surfaceArray)
   {
      totalArea += s.getLength() * s.getWidth();
   }

   if (totalArea <= numGallons * 400)
   {
      return true;
   }
   else
   {
```

## Unit 4 Progress Check: FRQ Part A

```
      return false;
   }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- call `getWidth` or `getLength` incorrectly

Responses will not earn the point if they

- use incorrect syntax for the condition
- fail to call `getWidth` or `getLength`

**Point 4**

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Calculates area of at least one element of the array

**Canonical Solution:**

```
public boolean hasEnoughPaint(int numGallons)
{
   double totalArea = 0.0;
   for (Surface s : surfaceArray)
   {
      totalArea += s.getLength() * s.getWidth();
   }

   if (totalArea <= numGallons * 400)
   {
      return true;
   }
   else
   {
      return false;
```

## Unit 4 Progress Check: FRQ Part A

```
    }
}
```

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- fail to calculate area of all surfaces
- return early

### Point 5

Select a point value to score the response.

To earn this point, responses must demonstrate the following:

- Returns `true` if there is enough paint and `false` otherwise (algorithm)

**Canonical Solution:**

```
public boolean hasEnoughPaint(int numGallons)
{
   double totalArea = 0.0;
   for (Surface s : surfaceArray)
   {
      totalArea += s.getLength() * s.getWidth();
   }

   if (totalArea <= numGallons * 400)
   {
      return true;
   }
   else
   {
      return false;
   }
}
```

## Unit 4 Progress Check: FRQ Part A

| 0 | 1 ✓ |
|---|---|

The response earns the point.

Responses can still earn the point even if they

- stop loop early or late due to incorrect bounds

Responses will not earn the point if they

- fail to return the result
- return an incorrect value due to early return